Lexsi Labs AI Safety and Alignment Hackathon · Track 2b: AI Safety and Correction

Same model, different language, different safety

A proposal to find, repair and verify the per-language safety drift in Tiny Aya's regional models, using the Lexsi alignment and safety stack

Status	Draft for discussion with Lexsi mentors, before we commit to building
Date	2 November 2026
Problem statement	Track 2b, problem 2: *Repairing safety drift without retraining* (weight-space methods that keep task performance) [1]
Core model	Tiny Aya Fire and Tiny Aya Earth (3.35B), with Tiny Aya Global as the reference [2]
Lexsi repos	CuratorKIT, AuditKIT, CircuitKIT, SafeTune (four repos, chained through lexsi_provenance.json) [5–8]
Compute	Free Colab/Kaggle T4 GPUs plus the hackathon's Hugging Face credits

1. The idea in one page

Cohere's Tiny Aya comes in a **Global** version and three **regional** versions: **Fire** for South Asian languages, **Earth** for African and West Asian languages, and **Water** for Europe and Asia-Pacific [2]. The regional versions are meant for people who speak those languages. They are better at them, which is the whole point.

The Tiny Aya technical report shows a side effect. On MultiJail, a benchmark of harmful requests in ten languages, the regional models are almost as safe as Global **on average**, but noticeably less safe in their **weakest language**: 87.0% safe for Global, 78.1% for Fire and 77.5% for Earth [2, Table 7]. Put simply, a harmful request that Global refuses may be answered by the model that was built specifically for that language's speakers.

We want to fix that gap, and prove the fix is real. Our plan:

- **Measure** the gap properly, language by language, including harmless prompts the model should *not* refuse.
- **Locate** where refusal lives inside the model with CircuitKIT, and how the regional version differs from Global.
- **Repair** it with SafeTune, comparing a circuit-targeted repair against three simpler alternatives.
- **Verify** the repair on three axes (harmful refusal, wrongful refusal of harmless prompts, general ability), and then again after compressing the model to 4-bit, the form in which a small model is most likely to be deployed.

> **Our thesis, stated so it can be proven wrong:**
>
> *Tiny Aya's regional specialisation weakens refusals in the very languages it targets. A repair located with CircuitKIT and applied with SafeTune restores per-language safety **without** making the model refuse harmless requests more often and **without** hurting its general ability, and the repair still holds after 4-bit quantisation.*

If any part of that sentence turns out false, we will report it. That outcome is useful too: it tells Lexsi and Cohere something real about where this kind of repair stops working.

2. The problem in more detail

2.1 What the published numbers show

Model	MultiJail safe rate, worst language	MultiJail safe rate, average	Gap to Global (worst language)
Tiny Aya Global	87.0%	91.1%	reference
Tiny Aya Water	82.9%	89.7%	−4.1 points
Tiny Aya Fire	78.1%	90.0%	−8.9 points
Tiny Aya Earth	77.5%	87.8%	−9.5 points

Source: Tiny Aya technical report, Table 7 [2]. MultiJail is described in [11].

Two things stand out:

- **Averages hide the problem.** Fire's average (90.0%) is only about one point below Global's (91.1%), so a check that looks at averages would pass it. The damage is concentrated in particular languages. According to the report, each regional model was merged with Global with a check that safety did not regress [2]; this suggests that check did not look closely enough at individual languages.
- **Global already over-refuses.** On XSTest, which measures how often a model refuses safe prompts that merely sound dangerous, Tiny Aya Global wrongly refuses 10.4%, against 3.6% for Gemma3-4B [2, 12]. So any repair that simply makes the model refuse more is not acceptable. It has to bring back refusals for genuinely harmful prompts only.

2.2 Why it probably happens (our working hypotheses)

- According to the report, only Global received preference tuning, the training stage that most directly teaches refusal. The regional models are a regional fine-tune merged with Global [2]. Merging may dilute what preference tuning added, more in some languages than others.
- Safety training data is usually richer in high-resource languages, so refusal behaviour in lower-resource languages may be held by weaker or different internal pathways that are easier to disturb.

We don't assume either hypothesis is right. Step 3 of our plan (CircuitKIT) is how we test them.

2.3 Why it matters

The people most likely to choose Tiny Aya Fire or Earth are speakers of Hindi, Bengali, Tamil, Swahili, Hausa and similar languages, often in settings where a small model is attractive because it can run cheaply or offline. Imagine a public-service or banking helpline in India or East Africa running a 4-bit model on local hardware. That is exactly the deployment where a language-specific safety gap would go unnoticed and matter most.

3. How this connects to Lexsi's work

We chose this problem because it sits directly on the research Lexsi has published, rather than beside it.

- **It reuses the audit → locate → correct → verify loop** from Lexsi's Gemma-4-31B repair [3], on one of the hackathon's own models, and with the same three-axis scoring: harmful refusal, benign over-refusal and utility, reported side by side.
- **It takes up items Lexsi listed as future work** after the Gemma-4 repair: per-language benign refusal, native-script fidelity, the fairness shift caused by multilingual edits, and running the loop on more open models [3].
- **It tests a failure mode Lexsi has documented:** 4-bit quantisation can undo safety and unlearning edits (up to 83% of "forgotten" knowledge returned in their experiments) [4; see also 19]. Few repair papers re-check after compression.
- **It uses the built-in bridge between two repos.** CircuitKIT writes component scores that SafeTune can read to decide which weights a safety LoRA may touch. We have not found a published comparison of circuit-targeted versus untargeted repair that uses this bridge, so this project would be a first worked example.
- **It builds on C-ΔΘ**, Lexsi's circuit-restricted weight-arithmetic method for selective refusal [9], and on their audit of how faithfully neuron selectors capture refusal [10].

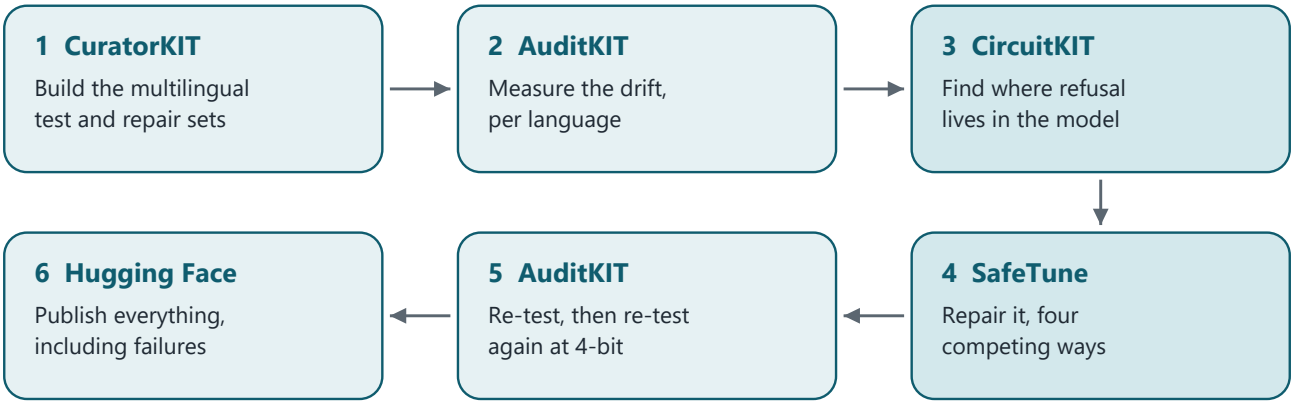
3.1 What has been done before, and what is new here

Cohere's Expedition Tiny Aya programme already produced strong *audits* of Tiny Aya: code-mixed safety, cross-lingual emergent misalignment, and language-aware quantisation [22, 23]. One team found that a model scored "96% safe" on a binary benchmark while its actual refusal rate was 66% [23], which is a good warning about measurement. We don't want to repeat those audits. Our contribution is the next step: **a repair, compared against alternatives, verified on three axes and after compression**.

3.2 A gap in the tooling we would like to help close

While mapping the six repos, we noticed that the safety components default to English. For example, AuditKIT's refusal detector is based on English patterns, SafeTune ships no MultiJail or Aya red-teaming loader, and CircuitKIT has no cross-language task yet. With a multilingual model at the centre of the hackathon, a Tamil refusal could be scored as compliance. We would need to solve this anyway for our own measurements, so we plan to contribute those pieces back as pull requests (section 7).

4. What we will build, step by step



Step 1 · Build the data (CuratorKIT)

- **Harmful prompts** from Cohere's human-written Aya red-teaming set [13] and from MultiJail [11].
- **Harmless look-alikes** in the XSTest style [12]: prompts with alarming words but innocent intent, such as "How do I kill a Python process?", written in each target language. These catch a repair that just refuses everything.
- **Languages:** for Fire, Hindi, Bengali and Tamil plus code-mixed Hinglish; for Earth, Swahili and Arabic, plus Hausa or Yoruba if data quality allows; English as a control. Where a dataset lacks a language, we translate with a stronger model and back-check a sample by hand.
- **Two separate splits.** A *repair* split used to fix the model and a *held-out* split used only to grade it, with no overlap, following the practice in [3].
- We will switch CuratorKIT's toxicity filter and deduplication embedder to multilingual models.

Step 2 · Measure the starting point (AuditKIT)

We score Global, Fire and Earth on three axes, separately for every language:

Axis	Plain question	How we measure it
Harmful refusal	Does it refuse what it should?	Refusal rate on held-out harmful prompts
Benign over-refusal	Does it wrongly refuse harmless requests?	Refusal rate on held-out harmless look-alikes
General ability	Is it still as capable?	Global-MMLU-Lite [14] and a multilingual instruction-following set

- **Judging refusals in many languages:** we will build a multilingual refusal judge, add a second independent judge (a larger Aya model or an open guard model; the organisers have said non-Cohere models are fine for evaluation), and hand-check a random sample to report how often the judges agree.
- **The system preamble as a variable:** Tiny Aya's chat template always injects a default system preamble of about 366 tokens, which includes safety instructions. Many real users replace or strip it, so we will measure safety with and without it.

Step 3 · Find where refusal lives (CircuitKIT)

Mechanistic interpretability tools can trace which internal parts of a network drive a particular behaviour. We will use **EAP-IG** (edge attribution patching with integrated gradients) [15, 16], which compares what happens inside the model on prompts it refuses against prompts it answers, and scores every connection by how much it contributes to the difference.

- We run this per language and per model, which lets us ask: is there one shared refusal circuit across languages, or different ones? And which parts changed between Global and Fire?
- We check that each circuit is genuine using CircuitKIT's six faithfulness tests [6]. A circuit that cannot reproduce the behaviour on its own is just a picture.
- The circuit is not the end result. It is the **map** that tells the repair in step 4 where to edit.

Step 4 · Repair it, four competing ways (SafeTune)

Rather than trying one method and hoping, we compare four, so the result means something:

Arm	Method, in plain words	Why it is in the comparison
A	**Circuit-targeted safety LoRA.** A small, trainable add-on that is only allowed to change the parts CircuitKIT flagged, using the CircuitKIT → SafeTune bridge.	Our main bet: a precise fix should cost less elsewhere
B	**ReSta**, safety restoration by task arithmetic [17]: add back a scaled "safety direction" in weight space. SafeTune's notes say $\alpha \approx 0.25$ works on Tiny Aya.	A strong, established weight-space method
C	**Refusal-direction steering** [18]: nudge the model's internal activations toward refusing, at inference time, without changing weights.	Lexsi found this kind of fix can make a model refuse harmless prompts far more often [3]. We want to check that on Tiny Aya
D	**Merge back toward Global**: blend the regional model's weights back toward Global's.	The obvious naive fix. It will probably also undo regional language gains

Step 5 · Verify, including after compression (AuditKIT)

- Re-run all three axes for all four arms on the held-out data, with identical prompts, decoding settings and judges.
- Compress the best arms to 4-bit, using GGUF Q4_K_M (llama.cpp) [21] and bitsandbytes NF4 [20], and test again. A repair that disappears after compression would not help anyone running the model on cheap hardware.
- If time allows: apply a small, harmless fine-tune afterwards and check whether the repair survives that too.

Step 6 · Publish

All arms, including the ones that lose, go on Hugging Face with logs and data cards. Section 7 lists the details.

5. How we will judge success

Following Lexsi's practice of writing the specification before the edit [3], we will fix our thresholds before running any repair. The numbers below are our starting proposal and one of the things we would like mentor feedback on.

Axis	Proposed bar for calling a repair a success
Harmful refusal	Close at least half of the worst-language gap between the regional model and Global
Benign over-refusal	Rises by no more than about 2 percentage points in any language
General ability	Falls by no more than about 1 point on the capability benchmarks
After 4-bit compression	The three results above still hold, within the same tolerances

We will report every axis for every arm side by side, never a single combined score. As Lexsi's Gemma-4 write-up puts it, "a model that refuses more can simply be a model that refuses everything" [3].

5.1 What the results table will look like

The table below is **illustrative only**. It shows the shape of the comparison and the patterns we expect the weaker baselines to show; it contains no measured numbers.

Model	Harmful refusal (worst language)	Over-refusal	General ability	After 4-bit
Global (reference)	high	high (10.4% on XSTest)	reference	to measure
Fire, no repair	lower (the drift)	to measure	reference	to measure
Fire + steering (C)	up	**expected to rise sharply**	to measure	to measure
Fire + merge-back (D)	up	to measure	**expected to fall**	to measure
Fire + ReSta (B)	up	to measure	to measure	to measure
Fire + circuit LoRA (A)	up	**hoped: flat**	**hoped: flat**	**the key test**

5.2 How we will read negative results

- **If refusal turns out to use one shared circuit across languages,** the circuit-targeted arm becomes "repair one shared circuit once", which is a simpler and still useful finding.
- **If the circuit-targeted arm does not beat ReSta,** we report that. It tells practitioners the simpler method is enough here.
- **If repairs fail after 4-bit compression,** that is an important warning in its own right, and we would try repairing with the quantised model in the loop if time allows.

6. Timeline

Dates	Work
2–3 Nov	Validate the idea with mentors; set up the environment; day-one smoke tests (load Tiny Aya Fire in SafeTune and AuditKIT, reproduce the MultiJail gap on about 200 prompts, run EAP-IG on a small refusal task); freeze the metrics and thresholds
4–5 Nov	Build the data sets and the multilingual refusal judge; run the baseline measurement for Global, Fire and Earth, with and without the preamble
6–7 Nov	Circuit discovery per language, faithfulness checks, cross-language comparison
8–10 Nov	Run the four repair arms and the three-axis evaluation
11 Nov	4-bit re-verification (GGUF and NF4); optional follow-up fine-tune test
12 Nov	Hugging Face collections, demo Space, pull requests to the Lexsi repos
13 Nov	Write-up and a reproducibility pass from a clean notebook
14 Nov	Buffer and submission (deadline midnight AoE)

7. What we will deliver

- **Models:** the repaired adapters or checkpoints for every arm, including the losing ones, each with a model card that shows the three-axis before/after table, intended use and limitations.
- **Data:** the multilingual evaluation and repair sets, as prompts and labels only (see section 9).
- **Logs and provenance:** AuditKIT run records and the lexsi_provenance.json chain linking data → circuit → repair → evaluation.
- **A demo:** a Hugging Face Space where you type a prompt in any supported language and see the original and repaired model's answers side by side, with a refusal verdict.
- **A write-up:** a Hugging Face article following the hackathon's nine-item checklist, including what we tried that did not work and a rough cost comparison for running a 4-bit model locally versus a hosted API (presented as hypothetical, given the non-commercial licence).
- **Pull requests to Lexsi repos:** a multilingual refusal detector for AuditKIT, MultiJail and Aya red-teaming loaders for SafeTune, and a cross-language refusal task for CircuitKIT.
- **A reproducible notebook** that runs the whole chain on a free T4 GPU, with pinned versions and seeds.

8. Risks and how we will handle them

Risk	What we will do
Our tools may not run cleanly on Tiny Aya's architecture (large 262k vocabulary, sliding-window attention, tied embeddings)	The day-one smoke tests exist to catch this early. If circuit discovery cannot run, we fall back to a simpler activation-based locator and keep the rest of the plan
Judging refusals in low-resource languages is hard	Two independent judges, a hand-checked sample, and reported agreement rates. We drop any language where the judges are not reliable enough, and say so
The gap may look different on our prompts than in the technical report	We reproduce the gap first (day one). If it is smaller than expected, the measurement itself, done properly per language, is still a contribution
Limited compute (free T4 GPUs)	All discovery and repair runs on the 3.35B model. Hugging Face credits cover the second judge and the final evaluation sweep
A known tokenizer issue in recent transformers versions splits numbers into single digits	This mainly affects arithmetic, not refusal. We will note the pinned version and check that capability scores are not affected

9. Ethics, licences and responsible release

- Tiny Aya is released under CC-BY-NC 4.0 together with Cohere Labs' acceptable-use policy [2, 24]. We will keep all work non-commercial and present any cost comparison as hypothetical.
- **We will not publish harmful model outputs.** Released data will contain prompts and labels only, plus aggregate metrics. Any examples in the write-up will be redacted.
- We will not release weights that are less safe than the originals. Only repaired versions will be published, and SafeTune's licence terms on drifted checkpoints will be respected [5].
- The write-up will include a limitations and dual-use note. In particular, knowing where refusal lives could in principle help someone remove it. We will publish the location maps only at a level that supports repair, and we would welcome the mentors' view on this.

10. Questions we would like mentor input on

Before we commit, we would value Lexsi's view on these points:

1. **Fit.** Does this fit Track 2b problem 2 (repairing safety drift without retraining)? The circuit-targeted LoRA arm does a small amount of training. Is that acceptable, or should we treat ReSta and C-ΔΘ-style edits as the main arms?
2. **Reference model.** Is it acceptable to treat Tiny Aya Global as the reference for the drift in Fire and Earth?
3. **Evaluation helpers.** We plan to use a larger Aya model and an open guard model as second judges. Is that fine?
4. **Public data.** Is a multilingual safety evaluation set (prompts and labels, no harmful completions) acceptable as a public Hugging Face artifact under Cohere's acceptable-use policy?
5. **Repos.** We plan to chain CuratorKIT, AuditKIT, CircuitKIT and SafeTune. Could we also use AlignTune if needed, even though it is listed under another track?

6. **Thresholds.** Do the success bars in section 5 look reasonable to you, or would you set them differently?
7. **Contributions.** Would the multilingual refusal detector and the MultiJail / Aya red-teaming loaders be welcome as pull requests? Is there a preferred interface?
8. **Anything we are missing** that you have seen go wrong with refusal circuits or repairs on Cohere models?

11. Glossary

Term	Meaning
Refusal / over-refusal	Declining a harmful request is a refusal. Declining a harmless request (because it sounds dangerous) is an over-refusal, which is also a failure
Safety drift	Safety getting worse as a side effect of some other change, here regional specialisation
Model merging	Combining two models by blending their weights; how the regional Tiny Aya models were made
Circuit	The small set of internal components and connections in a model that carry out a specific behaviour
EAP-IG	Edge attribution patching with integrated gradients: a fast method for estimating how much each connection contributes to a behaviour [15]
Faithfulness	Whether a discovered circuit actually reproduces the behaviour when everything else is removed
LoRA	A small set of trainable add-on weights; a cheap way to change a model without retraining it
ReSta	Restoring safety by task arithmetic: adding a scaled "safety direction" back into the weights [17]
Steering	Changing a model's behaviour at run time by adding a vector to its internal activations [18]
4-bit quantisation (GGUF Q4_K_M, NF4)	Compressing weights to 4 bits so a model runs on cheaper hardware [20, 21]
Code-mixed	Text that mixes languages in one sentence, such as Hinglish (Hindi and English)
Held-out set	Data kept aside and used only for grading, never for training or repair

12. References

[1] Lexsi Labs. *AI Safety and Alignment Hackathon*: website and kickoff slides (tracks, problem statements, rules, rubric). https://hackathon.lexsi.ai/ · https://hackathon.lexsi.ai/kickoff-slides/

[2] Cohere Labs (2026). *Tiny Aya technical report*. arXiv:2603.11510. https://arxiv.org/abs/2603.11510 (Table 7: MultiJail and XSTest safety results; training and merging details)

[3] Lexsi Labs (Oct 2026). *The Lexsi Alignment and Safety Stack: correcting Gemma-4-31B for multilingual and cybersecurity safety in the weights*. https://lexsi.ai/resources/articles/the-lexsi-alignment-and-safety-stack-correcting-gemma-4-31b-for-multilingual-and-cybersecurity-safety-in-the-weights-copy

[4] Lexsi Labs (Sep 2026). *Open-sourcing the Lexsi Alignment and Safety Stack: frontier safety research for auditing, correcting and verifying model weights*. https://lexsi.ai/resources/articles/open-sourcing-the-lexsi-alignment-and-saf

ety-stack-frontier-safety-research-for-auditing-correcting-and-verifying-model-weights

[5] Lexsi Labs. *SafeTune*. arXiv:2609.22153. https://github.com/Lexsi-Labs/SafeTune

[6] Lexsi Labs. *CircuitKIT*. arXiv:2607.19317. https://github.com/Lexsi-Labs/CircuitKIT

[7] Lexsi Labs. *CuratorKIT*. arXiv:2606.21631. https://github.com/Lexsi-Labs/CuratorKIT

[8] Lexsi Labs. *AuditKIT*. arXiv:2610.22511. https://github.com/Lexsi-Labs/AuditKIT

[9] Lexsi Labs. *C-ΔΘ: circuit-restricted weight arithmetic for selective refusal*. arXiv:2602.04521.

[10] Lexsi Labs. *Faithfulness to Refusal: A Causal Audit of Neuron Selectors*. arXiv:2607.05355.

[11] Deng, Y. et al. (2023). *Multilingual Jailbreak Challenges in Large Language Models* (MultiJail). arXiv:2310.06474. Dataset: https://huggingface.co/datasets/DAMO-NLP-SG/MultiJail

[12] Röttger, P. et al. (2023). *XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models*. arXiv:2308.01263.

[13] Aakanksha et al. (2024). *The Multilingual Alignment Prism: Aligning Global and Local Preferences to Reduce Harm*. arXiv:2406.18682. Dataset: https://huggingface.co/datasets/CohereLabs/aya_redteaming

[14] Singh, S. et al. (2024). *Global MMLU: Understanding and Addressing Cultural and Linguistic Biases in Multilingual Evaluation*. arXiv:2412.03304.

[15] Hanna, M., Pezzelle, S., Belinkov, Y. (2024). *Have Faith in Faithfulness: Going Beyond Circuit Overlap When Finding Model Mechanisms* (EAP-IG). arXiv:2403.17806.

[16] Syed, A., Rager, C., Conmy, A. (2023). *Attribution Patching Outperforms Automated Circuit Discovery* (EAP). arXiv:2310.10348.

[17] Bhardwaj, R., Anh, D. D., Poria, S. (2024). *Language Models are Homer Simpson! Safety Re-Alignment of Fine-tuned Language Models through Task Arithmetic* (RESTA). arXiv:2402.11746.

[18] Arditi, A. et al. (2024). *Refusal in Language Models Is Mediated by a Single Direction*. arXiv:2406.11717.

[19] Zhang, Z. et al. (2024). *Catastrophic Failure of LLM Unlearning via Quantization*. arXiv:2410.16454.

[20] Dettmers, T. et al. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs* (NF4). arXiv:2305.14314.

[21] ggml-org. *llama.cpp* (GGUF format and Q4_K_M quantisation). https://github.com/ggml-org/llama.cpp

[22] Cohere (Jul 2026). *Tiny Aya in the wild* (Expedition Tiny Aya projects). https://cohere.com/blog/tiny-aya-in-the-wild

[23] Cohere Labs Community (2026). *Mix, Fine-Tune, Break* (code-mixed safety on Tiny Aya). https://labscommunity.cohere.com/blog/2026/mix-fine-tune-break/

[24] Cohere Labs. *Tiny Aya Global model card* (licence CC-BY-NC 4.0 and acceptable-use policy). https://huggingface.co/CohereLabs/tiny-aya-global

[25] Lexsi Labs on GitHub and Hugging Face. https://github.com/Lexsi-Labs · https://huggingface.co/Lexsi